

C'est qui ce ZIG ?

Didier Plaindoux



@dplaindoux@functional.cafe







**Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !**



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées

Langage sans préprocesseur, ni macros



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées

Langage sans préprocesseur, ni macros

Langage avec un outillage avancé

Survol du langage ZIG



Types natifs

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, i128, u128, isize, usize

f16, f32, f64, f80, f128

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, i128, u128, isize, usize
f16, f32, f64, f80, f128

Compatibilité ABI avec C

c_char, c_short, c_ushort, c_int, c_uint, c_long etc.

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, i128, u128, isize, usize
f16, f32, f64, f80, f128

Compatibilité ABI avec C

c_char, c_short, c_ushort, c_int, c_uint, c_long etc.

Autres ...

bool, void, anyopaque, noreturn, type, anytype, anyerror

Tableaux

Tableaux

Tableau statique

```
const hello: [5]u8 = [_]u8{ 'h', 'e', 'l', 'l', 'o' };
```

Tableaux

Tableau statique

```
const hello: [5]u8 = [_]u8{ 'h', 'e', 'l', 'l', 'o' };
```

“Slice” (pointeur + taille)

```
const he: []const u8 = hello[0..2];
```

Tableaux

Tableau statique

```
const hello: [5]u8 = [_]u8{ 'h', 'e', 'l', 'l', 'o' };
```

“Slice” (pointeur + taille)

```
const he: []const u8 = hello[0..2];
```

Tableau dynamique avec `std.ArrayList`

Tableaux

Tableau statique

```
const hello: [5]u8 = [_]u8{ 'h', 'e', 'l', 'l', 'o' };
```

“Slice” (pointeur + taille)

```
const he: []const u8 = hello[0..2];
```

Tableau dynamique avec `std.ArrayList`

Vecteur (support SIMD)

```
const v1: @Vector(4, i32) = .{ 1, 2, 3, 4 };
```

Contrôle

Contrôle

```
if (condition) true_branch [else false_branch]
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

```
while(i < 10) { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

```
while(i < 10) { ... }
```

```
while (i < 10) : (i += 1) { ... }
```


Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

```
while(i < 10) { ... }
```

```
while (i < 10) : (i += 1) { ... }
```

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Type des paramètres et résultat obligatoire

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Type des paramètres et résultat obligatoire

Les paramètres sont passés par valeur

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Type des paramètres et résultat obligatoire

Les paramètres sont passés par valeur

- les valeurs primitives sont copiées et

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Type des paramètres et résultat obligatoire

Les paramètres sont passés par valeur

- les valeurs primitives sont copiées et
- pour les données composites cela dépend

Optionel ?T

Soit la valeur `null` soit une valeur de type `T`

Optionnel ?T

Soit la valeur **null** soit une valeur de type **T**

```
const a : ?u32 = ...;
```


Optionel ?T

Soit la valeur `null` soit une valeur de type `T`

```
const a : ?u32 = ...;
```

```
const value1 = a or else ...;
```

Optionel ?T

Soit la valeur `null` soit une valeur de type `T`

```
const a : ?u32 = ...;
```

```
const value1 = a or else ...;
```

```
const value2 = a or else unreachable; // panique si [null]
```

Optionel ?T

Soit la valeur `null` soit une valeur de type `T`

```
const a : ?u32 = ...;
```

```
const value1 = a or else ...;
```

```
const value2 = a or else unreachable; // panique si [null]
```

```
const value3 = a.?; // équivalent à [a or else unreachable]
```

Optionnel ?T

Soit la valeur `null` soit une valeur de type `T`

```
const a : ?u32 = ...;
```

```
const value1 = a or else ...;
```

```
const value2 = a or else unreachable; // panique si [null]
```

```
const value3 = a.?; // équivalent à [a or else unreachable]
```

```
if (a) |value| { ... } else { ... }
```

Optionnel ?T

Soit la valeur **null** soit une valeur de type **T**

```
const a : ?u32 = ...;
```

```
const value1 = a or else ...;
```

```
const value2 = a or else unreachable; // panique si [null]
```

```
const value3 = a.?; // équivalent à [a or else unreachable]
```

```
if (a) |value| { ... } else { ... }
```

Enumération

Ensemble restreint de valeurs nommées

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum { pique, trefle, carreau, coeur };
```

Enumération

Traitement par filtrage exhaustif

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum { pique, trefle, carreau, coeur };  
  
fn couleur(e: Enseigne) Couleur {  
    return switch (e) {  
        .pique, .trefle => .noir,  
        .carreau, .coeur => .rouge,  
    };  
}
```

Enumération

Colocaliser représentation et comportements

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum {  
    pique, trefle, carreau, coeur,  
  
    fn couleur(e: Enseigne) Couleur {  
        ...  
    }  
};  
// Enseigne.couleur(.pique) ou Enseigne.pique.couleur()
```


Enumération

Typage et référence au type courant

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum {  
    pique, trefle, carreau, coeur,  
  
    fn couleur(e: @This()) Couleur {  
        ...  
    }  
};  
// Enseigne.couleur(.pique) ou Enseigne.pique.couleur()
```

Erreur

Enumérations spécifiques dédiées aux erreurs

Erreur

Enumérations spécifiques dédiées aux erreurs

```
const A = error{ NotDir, PathNotFound };
```

Erreur

Enumérations spécifiques dédiées aux erreurs

```
const A = error{ NotDir, PathNotFound };
```

```
const B = error{ OutOfMemory, PathNotFound };
```

Erreur

Enumérations spécifiques dédiées aux erreurs

```
const A = error{ NotDir, PathNotFound };
```

```
const B = error{ OutOfMemory, PathNotFound };
```

```
const C = A || B;
```

```
// C = error{ OutOfMemory, NotDir, PathNotFound }
```

Erreur

Enumérations spécifiques dédiées aux erreurs

```
const A = error{ NotDir, PathNotFound };
```

```
const B = error{ OutOfMemory, PathNotFound };
```

```
const C = A || B;
```

```
// C = error{ OutOfMemory, NotDir, PathNotFound }
```

Pas de notion d'exceptions !

Fonction & Erreurs

Fonction & Erreurs

```
fn nom(p1: t1, ..., p1: tn) A!r { ... }
```


Fonction & Erreurs

```
fn nom(p1: t1, ..., p1: tn) A!r { ... }
```

```
fn nom(p1: t1, ..., p1: tn) error{OutOfMemory}!r { ... }
```

Fonction & Erreurs

```
fn nom(p1: t1, ..., p1: tn) A!r { ... }
```

```
fn nom(p1: t1, ..., p1: tn) error{OutOfMemory}!r { ... }
```

```
fn nom(p1: t1, ..., p1: tn) !r { ... }
```

Fonction & Erreurs

```
fn nom(p1: t1, ..., p1: tn) A!r { ... }
```

```
fn nom(p1: t1, ..., p1: tn) error{OutOfMemory}!r { ... }
```

```
fn nom(p1: t1, ..., p1: tn) !r { ... }
```

La remontée d'erreur est faite via `return ...`

Traitement des erreurs

Traitement des erreurs

```
const r1 = operation catch comportement_en_cas_d_erreur;
```

Traitement des erreurs

```
const r1 = operation catch comportement_en_cas_d_erreur;
```

```
const r2 = operation catch |e| comportement_en_cas_d_erreur;
```

Traitement des erreurs

```
const r1 = operation catch comportement_en_cas_d_erreur;
```

```
const r2 = operation catch |e| comportement_en_cas_d_erreur;
```

```
const r3 = operation catch |e| return e;
```

Traitement des erreurs

```
const r1 = operation catch comportement_en_cas_d_erreur;
```

```
const r2 = operation catch |e| comportement_en_cas_d_erreur;
```

```
const r3 = operation catch |e| return e;
```

```
const r3 = try operation; // Sucre syntaxique pour r3
```


Traitement des erreurs

```
const r1 = operation catch comportement_en_cas_d_erreur;
```

```
const r2 = operation catch |e| comportement_en_cas_d_erreur;
```

```
const r3 = operation catch |e| return e;
```

```
const r3 = try operation; // Sucre syntaxique pour r3
```

Fonction & Erreur

Un exemple complet

Fonction & Erreur

Un exemple complet

```
const MathError = error{DivideByZero};
```

Fonction & Erreur

Un exemple complet

```
const MathError = error{DivideByZero};

fn divide(a: f64, b: f64) MathError!f64 {
    if (b == 0) {
        return MathError.DivideByZero;
    }
}
```

Fonction & Erreur

Un exemple complet

```
const MathError = error{DivideByZero};

fn divide(a: f64, b: f64) MathError!f64 {
    if (b == 0) {
        return MathError.DivideByZero;
    }

    return a / b;
}
```

Erreur et Diagnostique

Type énuméré donc sans valeurs

Erreur et Diagnostique

Type énuméré donc sans valeurs

Approches possibles:

Erreur et Diagnostique

Type énuméré donc sans valeurs

Approches possibles:

- Remplacer **E!R** par un type algébrique comme **Result**

Erreur et Diagnostique

Type énuméré donc sans valeurs

Approches possibles:

- Remplacer **E!R** par un type algébrique comme **Result**
- Avoir un paramètre en écriture pour le diagnostique

Structure

Type produit hérité du langage C

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return Point { .x = x, .y = y };  
}
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return Point { .x = x, .y = y };  
}
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return .{ .x = x, .y = y };  
}
```

Structure

Colocaliser représentation et comportements

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) Point {  
        return .{ .x = x, .y = y };  
    }  
};
```

Structure

Typage et référence au type courant

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) @This() {  
        return .{ .x = x, .y = y };  
    }  
};
```


Structure

Structure

Tout fichier source est une Structure

Structure

Tout fichier source est une Structure

x: i32,

y: i32,

Structure

Tout fichier source est une Structure

```
x: i32,  
y: i32,
```

```
fn init(x: i32, y: i32) @This() {  
    return .{ .x = x, .y = y };  
}
```

Structure

Tout fichier source est une Structure

x: i32,

y: i32,

```
fn init(x: i32, y: i32) @This() {  
    return .{ .x = x, .y = y };  
}
```

Structure

Tout fichier source est une Structure

x: i32,

y: i32,

```
fn init(x: i32, y: i32) @This() {  
    return @This(){ .x = x, .y = y};  
}
```

Union

Type somme hérité du langage C

Union

Type somme hérité du langage C

```
const Number = union {  
    integer: i64,  
    float: f64,  
};
```


Union

Type somme hérité du langage C

```
const Number = union {  
    integer: i64,  
    float: f64,  
};
```

Soit un integer soit un float

Union

Manipulation sécurisée à la compilation

Union

Manipulation sécurisée à la compilation

```
const Number = union { integer: i64, float: f64, };
```

Union

Manipulation sécurisée à la compilation

```
const Number = union { integer: i64, float: f64, };
```

```
fn main() void {  
    const n = Number{ .integer = 42 };
```

Union

Manipulation sécurisée à la compilation

```
const Number = union { integer: i64, float: f64, };
```

```
fn main() void {  
    const n = Number{ .integer = 42 };  
    if (n.float == 0.0) {
```

Union

Manipulation sécurisée à la compilation

```
const Number = union { integer: i64, float: f64, };
```

```
fn main() void {  
    const n = Number{ .integer = 42 };  
    if (n.float == 0.0) {  
        ...  
    }  
}
```

Union

Manipulation sécurisée à la compilation

```
const Number = union { integer: i64, float: f64, };
```

```
fn main() void {  
    const n = Number{ .integer = 42 };  
    if (n.float == 0.0) {  
        ...  
    }  
}
```

error: access of union field 'float' while field 'integer' is active

Union

Manipulation sécurisée à l'exécution

Union

Manipulation sécurisée à l'exécution

```
const Number = union { integer: i64, float: f64, };
```

Union

Manipulation sécurisée à l'exécution

```
const Number = union { integer: i64, float: f64, };
```

```
fn integer(n: Number) i64 {  
    return n.integer;  
}
```

Union

Manipulation sécurisée à l'exécution

```
const Number = union { integer: i64, float: f64, };
```

```
fn integer(n: Number) i64 {  
    return n.integer;  
}
```

panic: access of union field 'integer' while field 'float' is active

Union Étiquetée

Union Étiquetée

Inspirée des langages fonctionnels

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite**

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite**
- Etiquetage Explicite**

Union Étiquetée

Inspirée des langages fonctionnels

- **Étiquetage Implicite**
- **Étiquetage Explicite**
- **Filtrage de Motif**

Union

Étiquetage implicite

Union

Étiquetage implicite

```
const Number = union(enum) { integer: i64, float: f64, };
```

Union

Étiquetage implicite

```
const Number = union(enum) { integer: i64, float: f64, };
```

```
fn zeroInteger() Number {  
    return .{ .integer = 0 };  
}
```

Union

Étiquetage implicite

```
const Number = union(enum) { integer: i64, float: f64, };
```

```
fn zeroInteger() Number {  
    return .{ .integer = 0 };  
}
```

```
fn zeroFloat() Number {  
    return .{ .float = 0 };  
}
```

Union

Étiquetage implicite

```
const Number = union(enum) { integer: i64, float: f64, };
```

```
fn zeroInteger() Number {  
    return .{ .integer = 0 };  
}
```

```
fn zeroFloat() Number {  
    return .{ .float = 0 };  
}
```

Union

Étiquetage explicite

Union

Étiquetage explicite

```
const Label = enum { entier, flottant };
```

Union

Étiquetage explicite

```
const Label = enum { entier, flottant };
```

```
const Number = union(Label) { entier: i64, flottant: f64, };
```


Union

Étiquetage explicite

```
const Label = enum { entier, flottant };
```

```
const Number = union(Label) { entier: i64, flottant: f64, };
```

```
fn zero(label: Label) Number {  
    return if (label == .entier) .{ .entier = 0 }  
        else .{ .flottant = 0 };  
}
```

Union

Étiquetage explicite

```
const Label = enum { entier, flottant };
```

```
const Number = union(Label) { entier: i64, flottant: f64, };
```

```
fn zero(label: Label) Number {  
    return if (label == .entier) .{ .entier = 0 }  
        else .{ .flottant = 0 };  
}
```

Union

Cas du Filtrage de Motif

```
const Number = union(enum) {
```

Union

Cas du Filtrage de Motif

```
const Number = union(enum) {  
    entier: i64, flottant: f64, };
```

```
fn increment(number: Number) Number {
```

Union

Cas du Filtrage de Motif

```
const Number = union(enum) {  
    entier: i64, flottant: f64, };
```

```
fn increment(number: Number) Number {  
    return switch (number) {
```

Union

Cas du Filtrage de Motif

```
const Number = union(enum) {  
  entier: i64, flottant: f64, };  
  
fn increment(number: Number) Number {  
  return switch (number) {  
    .entier => |n| .{ .entier = n + 1 },
```

Union

Cas du Filtrage de Motif

```
const Number = union(enum) {  
    entier: i64, flottant: f64, };  
  
fn increment(number: Number) Number {  
    return switch (number) {  
        .entier => |n| .{ .entier = n + 1 },  
        .flottant => |f| .{ .flottant = f + 1 },  
    };  
}
```

Union

Colocaliser représentation et comportements

```
const Number = union(enum) {  
    entier: i64, flottant: f64,  
  
    fn increment(number: Number) Number {  
        return ...  
    }  
};
```


Union

Typage et référence au type courant

```
const Number = union(enum) {  
    entier: i64,  
    flottant: f64,  
  
    fn increment(self: @This()) @This() {  
        return ...  
    }  
};
```

Union

Cas des types récurifs

Union

Cas des types récurifs

```
const Naturl = union(enum) {  
    Zero,  
    Succ: Naturl,  
};
```

Union

Cas des types récurifs

```
const Naturel = union(enum) {  
    Zero,  
    Succ: Naturel,  
};
```

```
error: type 'Naturel' depends on itself for field declared here  
    Succ: Naturel,  
          ^~~~~~
```

Union

Cas des types récurifs

```
const Naturel = union(enum) {  
    Zero,  
    Succ: Naturel,  
};
```

```
error: type 'Naturel' depends on itself for field declared here  
    Succ: Naturel,  
          ^~~~~~
```

Il faut passer par les pointeurs !

Pointeur

Pointeur

Définition de type : $*T$ jamais nul sauf si T est optionnel

Pointeur

Définition de type : $*T$ jamais nul sauf si T est optionnel

Syntaxe : $\&$ pour l'adresse et $.*$ pour déréférencer

Pointeur

Définition de type : `*T` jamais nul sauf si `T` est optionnel

Syntaxe : `&` pour l'adresse et `.*` pour déréférencer

Implicite : `.*` optionnel lors d'accès (fonction ou champ)

Pointeur

Définition de type : $*T$ jamais nul sauf si T est optionnel

Syntaxe : $\&$ pour l'adresse et $.*$ pour déréférencer

Implicite : $.*$ optionel lors d'accès (fonction ou champ)

Arithmétique indirecte : par coercion de type

Pointeur

Définition de type : `*T` jamais nul sauf si `T` est optionnel

Syntaxe : `&` pour l'adresse et `.*` pour déréférencer

Implicite : `.*` optionel lors d'accès (fonction ou champ)

Arithmétique indirecte : par coercion de type

```
var x: i32 = 1234;
```

```
const ptr = &x;
```

```
ptr.* = 5678; // Modification de la valeur
```

```
ptr += 1; // Erreur de compilation
```

Gestion de la mémoire dans ZIG



Pile ou Tas ?

La Pile (Stack)

Taille connue et durée de vie liée au bloc de base

Portée : Destruction dès que l'on sort du bloc de base

Performance : Allocation efficace mais taille limitée

Contrainte : La taille doit être connue à la compilation

Allocation sur la Pile

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };
```

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };
```

```
pub fn exemple() void {  
    const p1 = Point { .x = 0, .y = 0};  
}
```


Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };

pub fn exemple() void {
    const p1 = Point { .x = 0, .y = 0 };
    {
        const p2 = Point { .x = 42, .y = 42 };
        ...
    } // -> p2 fin de portée donc libéré
```

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };

pub fn exemple() void {
    const p1 = Point { .x = 0, .y = 0 };
    {
        const p2 = Point { .x = 42, .y = 42 };
        ...
    } // -> p2 fin de portée donc libéré
    ...
} // -> p1 fin de portée donc libéré
```

Pile ou Tas ?

Le Tas (Heap)

Taille dynamique ou durée de vie indépendante de la pile

Philosophie Zig : Pas d'allocateur global caché

Passage d'allocateur : Allocator comme paramètre

Contrainte : Libération explicite (cf. fuite mémoire)

Allocation sur le Tas

Oui mais lequel ?

Allocation sur le Tas

Oui mais lequel ?

- DebugAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**
- **PageAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**
- **PageAllocator**
- ...

Allocation sur un tas

Allocation sur un tas

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
  
    const allocator = heap.allocator();
```

Allocation sur un tas

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
  
    const allocator = heap.allocator();  
  
    const array = try allocator.alloc(u8, 10);  
    ...  
}
```

Désallocation

Garantir la libération en sortie de bloc

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};
```

Désallocation

Garantir la libération en sortie de bloc

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
}
```

Désallocation

Garantir la libération en sortie de bloc

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const array = try allocator.alloc(u8, 10);
```

Désallocation

Garantir la libération en sortie de bloc

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const array = try allocator.alloc(u8, 10);  
    defer allocator.free(array);  
}
```


Désallocation

Garantir la libération en sortie de bloc

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const array = try allocator.alloc(u8, 10);  
    defer allocator.free(array);  
    ...  
}
```

Fuite mémoire

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
    const array = try allocator.alloc(u8, 10);  
    ...  
}
```

Fuite mémoire

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
    const array = try allocator.alloc(u8, 10);  
    ...  
}
```

error(DebugAllocator): memory address 0x10caa0000 leaked:
 const array = try allocator.alloc(u8, 10);
 ^

“Use-After-Free”

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();
```

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);
```

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
}
```

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
  
    score_ptr.* = 666; // Use-After-Free here
```


“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
  
    score_ptr.* = 666; // Use-After-Free here  
}
```

“Use-After-Free”

“Use-After-Free”

Execution avec le DebugAllocator :

Segmentation fault at address 0x109a20000

use-after-free.zig:13:5: 0x1010c10c1 in main

```
    score_ptr.* = 666;
```

^

“Use-After-Free”

Execution avec le DebugAllocator :

```
Segmentation fault at address 0x109a20000  
use-after-free.zig:13:5: 0x1010c10c1 in main  
    score_ptr.* = 666;  
    ^
```

Solutions idiomatiques :

- utilisation du defer ou
- typer avec un optionel et mettre le pointeur à null.

“Stack Buffer Overflow”

Écriture en dehors de la structure prévue

```
fn foo(input: []const u8) void {  
    var buffer: [8]u8 = undefined;  
    @memcpy(buffer[0..], input);  
}
```

```
pub fn main() void {  
    foo("Plus de 8 caractères!"); // 22 caractères  
}
```

“Stack Buffer Overflow”

Execution interrompue

```
thread 1332888 panic: ... arguments have non-equal lengths
stack-buffer-overflow.zig:5:19: 0x10230955f in foo
@memcpy(buffer[0..], input);
                ^
stack-buffer-overflow.zig:9:8: 0x102309314 in main
foo("Plus de 8 caractères!");
```

Détection des débordements (modes de compilation)

Voir aussi Stack canaries, ASLR et DEP/NX.

Méta-programmation en ZIG



Comptime vs. Runtime

Comptime vs. Runtime

Code exécuté à la compilation

Systeme de type strict, duck typing avec anytype

Comptime vs. Runtime

Code exécuté à la compilation

Système de type strict, duck typing avec anytype

Code exécuté au runtime

Système de type strict uniquement

Type comme Valeur

Type comme Valeur

Citoyen de 1ère classe à la compilation

Type comme Valeur

Citoyen de 1ère classe à la compilation

Évaluation à la compilation via comptime

Type comme Valeur

Citoyen de 1ère classe à la compilation

Évaluation à la compilation via comptime

Fonction génératrice de type

Type comme valeur

```
const Nat: type = union(enum) {  
    Zero, Succ: *const @This(),  
  
    fn toInt(self: *const @This()) u32 {  
        return switch(self) {  
            .Zero => 0,  
            .Succ => |p| 1 + p.toInt(),  
        };  
    }  
}
```

Type comme valeur

Type évalué à la compilation

Type comme valeur

Type évalué à la compilation

```
fn Integer(comptime unsigned: bool) type {
```

Type comme valeur

Type évalué à la compilation

```
fn Integer(comptime unsigned: bool) type {  
    if (b) {  
        return u64;  
    } else {  
        return f64;  
    }  
}
```

Type comme valeur

Type évalué à la compilation

```
fn Integer(comptime unsigned: bool) type {  
    if (b) {  
        return u64;  
    } else {  
        return f64;  
    }  
}
```

Type paramétrique ou Universel

Type évalué à la compilation

Type paramétrique ou Universel

Type évalué à la compilation

```
fn Optionel(comptime T: type) type {
```

Type paramétrique ou Universel

Type évalué à la compilation

```
fn Option1(comptime T: type) type {  
    return union(enum) {  
        None, Some: T,
```

Type paramétrique ou Universel

Type évalué à la compilation

```
fn Optionel(comptime T: type) type {  
    return union(enum) {  
        None, Some: T,  
  
        fn pure(t:T) @This() {  
            return .{ .Some = t };  
        }  
    };  
}
```

Déclaration et Génération de types

$\text{AddU32} = (\text{u32}, \text{u32}) \rightarrow \text{u32}$

```
const AddU32: type = fn(u32, u32) u32;
```

$\text{Optional} : \forall A : * . A \rightarrow ? A$

```
const Optionel: type = fn(comptime A: type) ?A;
```


Déclaration et Génération de types

$\text{AddU32} = (\text{u32}, \text{u32}) \rightarrow \text{u32}$

```
const AddU32: type = fn(u32, u32) u32;
```

$\text{Optional} : \forall A : * . A \rightarrow ? A$

```
const Optionel: type = fn(comptime A: type) ?A;
```

error: use of undeclared identifier 'A'

```
const Optionel: type = fn(comptime A: type) ?A
                                     ^
```

Déclaration et Génération de types

$\text{AddU32} = (\text{u32}, \text{u32}) \rightarrow \text{u32}$

```
const AddU32: type = fn(u32, u32) u32;
```

$\text{Optional} : \forall A : * . A \rightarrow ? A$

```
const Optionel: type = fn(comptime A: type) ?A;
```

error: use of undeclared identifier 'A'

```
const Optionel: type = fn(comptime A: type) ?A
                                     ^
```

```
fn Optionel(comptime A: type) type { return ?A; }
```

Type abstrait ou Existential

Type abstrait ou Existential

Encodage via une continuation et des $\forall$

Type abstrait ou Existential

Encodage via une continuation et des $\forall$

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {  
    return struct {  
        pub fn pack(X:type, value:T(X)) type {
```


$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {  
  return struct {  
    pub fn pack(X:type, value:T(X)) type {  
      return struct {  
        // anytype at comptime capture  $\forall x.T(x) \rightarrow y$   
        pub fn unpack(Y:type, cont:anytype) Y {  
          return cont(X, value);  
        }  
      };  
    }  
  };
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {  
  return struct {  
    pub fn pack(X:type, value:T(X)) type {  
      return struct {  
        // anytype at comptime capture  $\forall x.T(x) \rightarrow y$   
        pub fn unpack(Y:type, cont:anytype) Y {  
          return cont(X, value);  
        }  
      };  
    }  
  };  
}
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Optionel(comptime X:type) type { return ?X; }
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Optionel(comptime X:type) type { return ?X; }  
fn run(comptime X:type, value: Optionel(X)) void { ... }
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Optionel(comptime X:type) type { return ?X; }  
fn run(comptime X:type, value: Optionel(X)) void { ... }
```

```
fn main() void {  
    const existential = Exists(Optionel).pack(i32, null);
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Optionel(comptime X:type) type { return ?X; }  
fn run(comptime X:type, value: Optionel(X)) void { ... }  
  
fn main() void {  
    const existential = Exists(Optionel).pack(i32, null);  
    existential.unpack(void, run);  
}
```

$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Optionel(comptime X:type) type { return ?X; }  
fn run(comptime X:type, value: Optionel(X)) void { ... }  
  
fn main() void {  
    const existential = Exists(Optionel).pack(i32, null);  
    existential.unpack(void, run);  
}
```


$$\exists x.T(x) \stackrel{\text{def}}{=} \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Optionel(comptime X:type) type { return ?X; }  
fn run(comptime X:type, value: Optionel(X)) void { ... }  
  
fn main() void {  
    const existential = Exists(Optionel).pack(i32, null);  
    existential.unpack(void, run);  
}
```

Applicable uniquement à la compilation !

Conception logiciel et ZIG



Programmation Orientée Donnée

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) @This() {  
        return .{ .x = x, .y = y};  
    }  
  
    fn move(self: @This(), dx: i32, dy: i32) @This() {  
        return .{ .x = self.x + dx, .y = self.y + dy};  
    }  
};
```

Programmation Orientée Donnée

Programmation Orientée Donnée

```
fn main() void {  
    const zero = Point.init(0, 0);  
}
```

Programmation Orientée Donnée

```
fn main() void {  
    const zero = Point.init(0, 0);  
  
    // Classical functional call  
    const point1 = Point.move(zero, 42, 42);  
}
```

Programmation Orientée Donnée

```
fn main() void {  
    const zero = Point.init(0, 0);  
  
    // Classical functional call  
    const point1 = Point.move(zero, 42, 42);  
  
    // DOT syntax (syntactic sugar)  
    const point2 = zero.move(42, 42);  
}
```

Programmation Orientée Donnée

```
fn main() void {  
    const zero = Point.init(0, 0);  
  
    // Classical functional call  
    const point1 = Point.move(zero, 42, 42);  
  
    // DOT syntax (syntactic sugar)  
    const point2 = zero.move(42, 42);  
}
```


Programmation Orientée Donnée

Types sommes et Filtrage de motif

```
const Number = union(enum) {  
    entier: i64, flottant: f64,  
  
    fn increment(self: @This()) @This() {  
        return switch (number) {  
            .entier => |n| .{ .entier = n + 1 },  
            .flottant => |f| .{ .flottant = f + 1 },  
        };  
    }  
};
```

Programmation par Abstraction

Programmation par Abstraction

Encodage à l'aide de table virtuelle

Programmation par Abstraction

Encodage à l'aide de table virtuelle ~ DIY

Programmation par Abstraction

Encodage à l'aide de table virtuelle ~ DIY

```
const Figure: type = struct {  
    v_impl: *anyopaque,  
    v_surface: *const fn (self: *anyopaque) f64,
```

Programmation par Abstraction

Encodage à l'aide de table virtuelle ~ DIY

```
const Figure: type = struct {  
    v_impl: *anyopaque,  
    v_surface: *const fn (self: *anyopaque) f64,  
  
    pub fn surface(self: @This()) f64 {  
        return self.v_surface(self.v_impl);  
    }  
}
```

Programmation par Abstraction

Encodage à l'aide de table virtuelle ~ DIY

```
const Figure: type = struct {  
    v_impl: *anyopaque,  
    v_surface: *const fn (self: *anyopaque) f64,  
  
    pub fn surface(self: @This()) f64 {  
        return self.v_surface(self.v_impl);  
    }  
  
    pub inline fn from(impl: anytype) @This() { ... }
```

Programmation par Abstraction

Encodage à l'aide de table virtuelle ~ DIY

```
const Figure: type = struct {  
    v_impl: *anyopaque,  
    v_surface: *const fn (self: *anyopaque) f64,  
  
    pub fn surface(self: @This()) f64 {  
        return self.v_surface(self.v_impl);  
    }  
  
    pub inline fn from(impl: anytype) @This() { ... }  
};
```


Programmation par Abstraction

Proposition d'une mise en œuvre

Programmation par Abstraction

Proposition d'une mise en œuvre

```
const Cercle = struct {  
    rayon: f64,  
  
    pub fn surface(self: @This()) f64 {  
        return std.math.pi * self.rayon * self.rayon;  
    }  
};
```

Programmation par Abstraction

Proposition d'une mise en œuvre

```
const Cercle = struct {  
    rayon: f64,  
  
    pub fn surface(self: @This()) f64 {  
        return std.math.pi * self.rayon * self.rayon;  
    }  
};
```

Instanciación : `Figure.from(&Cercle{ .rayon = 10 })`

Programmation par Modules

Programmation par Modules

Definition de la signature de module Stack

Programmation par Modules

Definition de la signature de module **Stack**

```
fn Stack(comptime T:fn (type) type, comptime A:type) type {  
    return struct {  
        create: fn () callconv(."inline") T(A),  
        push: fn (Allocator, A, T(A)) anyerror!T(A),  
        peek: fn (T(A)) ?A,  
        pop: fn (Allocator, T(A)) Pair(?A, T(A)),  
    };  
}
```

Programmation par Modules

Programmation par Modules

Mise en oeuvre avec les `List`

Programmation par Modules

Mise en oeuvre avec les List

```
fn StackList(comptime A: type) type {  
    return struct {  
        inline fn create() List(A) {  
            return .nil();  
        }  
        ...  
    };  
}
```

Programmation par Modules

DSL avec une pincée de métacompilation

Programmation par Modules

DSL avec une pincée de métacompilation

```
fn implUsing(comptime T: type, comptime I: type) S {  
    var result: T = undefined;
```

Programmation par Modules

DSL avec une pincée de métacompilation

```
fn implUsing(comptime T: type, comptime I: type) S {  
    var result: T = undefined;  
  
    const target_info = @typeInfo(T).@"struct"; // unsafe  
    inline for (target_info.fields) |f| {  
        @field(result, f.name) = @field(I, f.name);  
    }  
}
```

Programmation par Modules

DSL avec une pincée de métacompilation

```
fn implUsing(comptime T: type, comptime I: type) S {  
    var result: T = undefined;  
  
    const target_info = @typeInfo(T).@"struct"; // unsafe  
    inline for (target_info.fields) |f| {  
        @field(result, f.name) = @field(I, f.name);  
    }  
  
    return result;  
}
```

Programmation par Modules

Mise en pratique

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
const s = implUsing(Stack(List, u32), StackList(u32));  
const stack = try s.push(allocator, 1, S.create());  
defer s.deinit(allocator, stack);
```


Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
const s = implUsing(Stack(List, u32), StackList(u32));  
const stack = try s.push(allocator, 1, S.create());  
defer s.deinit(allocator, stack);
```

```
try std.testing.expectEqual(1, s.peek(stack));
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
const s = implUsing(Stack(List, u32), StackList(u32));  
const stack = try s.push(allocator, 1, S.create());  
defer s.deinit(allocator, stack);
```

```
try std.testing.expectEqual(1, s.peek(stack));
```

Programmation par Modules

Est-ce pertinent ?

Programmation par Modules

Est-ce pertinent ?

```
const s = implUsing(Stack(List, u32), StackList(u32));  
// Expose le type interne ^^^^
```

Programmation par Modules

Est-ce pertinent ?

```
const s = implUsing(Stack(List, u32), StackList(u32));  
// Expose le type interne ^^^^
```

Abstraction via type existentiel limitée à comptime !

Programmation par Modules

Est-ce pertinent ?

```
const s = implUsing(Stack(List, u32), StackList(u32));  
// Expose le type interne ^^^^
```

Abstraction via type existentiel limitée à comptime !

Utiliser les vtable et les types opaques en runtime

Programmation Fonctionnelle

Programmation Fonctionnelle

Pas de fonction anonyme

Programmation Fonctionnelle

Pas de fonction anonyme

Pas de fermeture (closure)

Programmation Fonctionnelle

Pas de fonction anonyme

Pas de fermeture (closure)

Ordre supérieur

- **Fonction qui manipule des fonctions**
- **Fonction qui retourne une fonction**

 **ZIG et C**



Utiliser des librairies C

Importation Intuitive

Utiliser des librairies C

Importation Intuitive

```
const ncurses = @cImport({  
    @cInclude("ncurses.h");  
});
```

Utiliser des librairies C

Importation Intuitive

```
const ncurses = @cImport({  
    @cInclude("ncurses.h");  
});  
  
pub fn main() !void {  
    _ = ncurses.initscr();  
    defer _ = ncurses.endwin();  
    ...  
}
```

Mise à disposition à C

Exportation avec les types C dans Zig

```
extern fn min(a: c_int, b: c_int) callconv(.C) c_int {  
    return if (a < b) a else b;  
}
```

Mise à disposition à C

Exportation avec les types C dans Zig

```
extern fn min(a: c_int, b: c_int) callconv(.C) c_int {  
    return if (a < b) a else b;  
}
```

Declaration externe en C (classique)

```
extern int min(int a, int b);
```


Outillage  ZIG



Production de binaires

Production de binaires

Une seule commande sans dépendance

Production de binaires

Une seule commande sans dépendance

Embarque les compilateurs C et C++

Production de binaires

Une seule commande sans dépendance

Embarque les compilateurs C et C++

Traduction C vers Zig gratuite

Production de binaires

Une seule commande sans dépendance

Embarque les compilateurs C et C++

Traduction C vers Zig gratuite

Cross-Compilation native (incluant WASM)

Production de binaires

Une seule commande sans dépendance

Embarque les compilateurs C et C++

Traduction C vers Zig gratuite

Cross-Compilation native (incluant WASM)

Réécriture du Linker (ni de ld ni de lld)

Réécriture et enrobage de la Libc

Le Système de Build

Le Système de Build

Tout en Zig via un fichier `build.zig`

Le Système de Build

Tout en Zig via un fichier `build.zig`

Gestionnaire de paquets via `build.zig.zon`

Le Système de Build

Tout en Zig via un fichier `build.zig`

Gestionnaire de paquets via `build.zig.zon`

API riche

- Générer du code
- Exécuter des tests

Epilogue



Pourquoi choisir ZIG ?

Zig se positionne comme une alternative à C

Interopérabilité “triviale” avec le C

Pas de flux de contrôle caché

Maîtrise totale de la mémoire et du matériel

Puissance de Comptime vs. Préprocesseur

Outillage moderne

Quelques liens

 **Site Officiel**

 **Pourquoi Zig ? (Vidéo)**

 **Forum Zig**

 **Koan Zig**

 **Zig & la recherche**



C'est qui ce gonze ?

Système Distribué et modèle Acteur (Akawan Kaptngo)

Système Expert en Prolog (Crédifix)

Fonctions et Calcul des Ambiants (Ephel)

Libraries OCaml avec des amis (CargoCut)

Colophon

Présentation élaborée avec l'aide de Typst

C'est qui ce ZIG ?



Dépôt



Feedback

